SUMMER INTERNSHIP REPORT

PROJECT TITLE: SLAM

GUIDE PROFESSOR: RAJBABU VELMURUGAN

Submitted by

ALDEN J JOSEPH

AJAY G IYER

ACKNOWLEDGEMENTS

We would like to express our sincere thanks to **Prof. RAJBABU VELMURUGAN** for his helpful suggestions and guidance throughout the course of our summer internship. We have been extremely fortunate to get a chance to work under him for two months. With his help, we have developed a better understanding of Mapping and Localisation.

We would like to express our appreciation to the guidance of **Prof. KUMAR APPAIAH**. Which was also helpful in our understanding of the topic.

Also, we would like to thank the students in Control lab of the department. Whom were helpful in providing us with a bot to work with for our project, and in helping us with the understanding of the bot.

Last but not the least; We would like to thank our family, for their endless support and encouragement.

CONTENTS

1. ACKNOWLEDGEMENTS 2
2. SUMMARY 4
3. INTRODUCTION 5
4. SIMULATION IN MATLAB 11
5. REAL LIFE NAVIGATION OF THE BOT 42
6. CONCLUSION 55
7. REFERENCES 56

SUMMARY

The major focus of our project was to understand the concept of SLAM (Simultaneous Localisation And Mapping) and to learn in detail about the basic type of SLAM called the EKF SLAM (Extended Kalman Filter SLAM).

To understand the concept of SLAM, we went through a series of online lectures and a few ppts and a book which was helpful to understand the basics of robotics.

To further increase our understanding on EKF SLAM, we learnt a MATLAB code and created a simulation of a bot navigating through a fixed path and simultaneously mapping the surroundings.

We even modified the code to make the both navigate on its own depending on the landmarks on the map.

Taking it a step further, we tried coding SLAM into a bot and helping it to navigate through a room. Further coding can be done to map the area and storing it in the bot to help it move around.

The motive behind the learning of SLAM was that it is a very useful concept used to make bots autonomous and to navigate terrains which are not humanly accessible and get a map of the area. It is also can be in everyday uses. The understanding of the basic SLAM can later be used to further learn in depth about SLAM and implement even better algorithms.

The report is divided into three sections. The first section consists of the introduction of SLAM and EKF SLAM. The second section explains the simulation part of the project and the results obtained from it. The last section deals with the real life implementation on the bot.

SECTION I

INTRODUCTION

What is SLAM?

SLAM is Simultaneous Localisation And Mapping, as the name goes it does localisation and mapping at the same time. Localisation is inferring the position from a given map which helps us in navigating through it. Whereas mapping is creating a map from the observations obtained from navigating through the area. Hence SLAM is learning a map and locating the robot simultaneously.

SLAM can be used in various fields from places like home to the outer space. It is used in lawn mowers, etc. meanwhile even in reef monitoring in underwater and exploration of mines which might not be stable enough for a human to traverse.

What is EKF SLAM?

As we got to know about SLAM we understand that it like a problem which needs to be tackled to make robots autonomous. Hence numerous number of algorithms have been developed to tackle this. The basic algorithm to do so is the EKF SLAM.

This algorithm takes a probabilistic approach to the problem, the state of the robot is taken as a Gaussian distribution due to the uncertainties created in its state due to the number of real life errors that occur. For eg: When you ask a bot to move 5 m forward it gives the data to its wheel and moves forward. The robot thinks it moved 5 m forward but due to wear and tear of the wheel and the overuse of the motor the wheel would have turned such that it just moved 4.98 m forward or 5.02 m forward. So, when you get back this data you get an uncertainty in its position and thus is it taken as a Gaussian distribution. Hence it also gives out a Gaussian distribution as output which is the next state of the robot.

EKF is a type of Bayes filter use for the localisation. To understand more about EKF we need to know about Bayes filter.

Bayes Filter

To understand anything further we need to understand some terminology first. There is two kind of data, one is given data which consists of the robot's controls and initial position of the bot. The other data is the data obtained

from the bot as it undergoes SLAM, which are the map of the area and the robot's next position.

Let the controls of the bot be represented as $\mathbf{U_{1:k}}$ which means the controls given from the start of time till time/state K.

Let the state of the bot be represented as $\mathbf{X_{1:k}}$ which means the state of the bot from start of time after the initial state till time K

Let the observations obtained from the bot sensors be represented as $\mathbf{Z_{1:k}}$ which means the observations obtained from the start of time till time/state K.

Bayes filter works on the principle of Bayes theorem and Markov Assumption and it is considered as a recursive function to determine the next state from the previous state and the observations obtained and the controls given so far.

$$Bel(x_t) = P(x_t \mid u_1, z_1 \ldots, u_t, z_t)$$

Bayes
$$= \eta \; P(z_t \mid x_t, u_1, z_1, \ldots, u_t) \; P(x_t \mid u_1, z_1, \ldots, u_t)$$

Markov
$$= \eta \; P(z_t \mid x_t) \; P(x_t \mid u_1, z_1, \ldots, u_t)$$

Total prob.
$$= \eta \; P(z_t \mid x_t) \int P(x_t \mid u_1, z_1, \ldots, u_t, x_{t-1}) \; P(x_{t-1} \mid u_1, z_1, \ldots, u_t) \; dx_{t-1}$$

Markov
$$= \eta \; P(z_t \mid x_t) \int P(x_t \mid u_t, x_{t-1}) \; P(x_{t-1} \mid u_1, z_1, \ldots, u_t) \; dx_{t-1}$$

Markov
$$= \eta P(z_t \mid x_t) \int P(x_t \mid u_t, x_{t-1}) P(x_{t-1} \mid u_1, z_1, \ldots, z_{t-1}) \; dx_{t-1}$$

$$= \eta \; P(z_t \mid x_t) \int P(x_t \mid u_t, x_{t-1}) \; Bel(x_{t-1}) \; dx_{t-1}$$

The Bayes function is further broken down into two different steps called the prediction step and the correction step, which accounts all the noises that are present in the observations.

- Prediction

$$\overline{bel}(x_t) = \int p(x_t \mid u_t, x_{t-1})\, bel(x_{t-1})\, dx_{t-1}$$

- Correction

$$bel(x_t) = \eta\, p(z_t \mid x_t)\, \overline{bel}(x_t)$$

Extended Kalman filter

As the name suggests it is better version of Kalman filter which is also a Bayes filter, it was found that Kalman filter algorithm doesn't work for nonlinear functions and the fact that most of the robot's system are nonlinear in nature and hence Kalman filter was adopted.

EKF also works on the principle of Bayes filter. It linearizes the nonlinear function according to Taylor series expansion and proceeds further on the same way as the Bayes filter to determine the position

Consider your state at t be given as a nonlinear dependency of its previous state and current state control

$$x_t = g(u_t, x_{t-1})$$

And the observations observed also as a nonlinear dependency of its current state

$$z_t = h(x_t)$$

To go further, you need to understand two more terminology, the mean and the covariance matrix
The mean is the mean point of the state as the current state is Gaussian distribution. The covariance matrix gives the uncertainty of the Gaussian distribution and hence the overall matrix gives the uncertainty of various landmarks (various objects in the map are considered as landmarks) with the robot's location and other landmarks observed by the bot.

Hence, the Taylor expansion of the nonlinear functions is given as

- **Prediction:**

$$g(u_t, x_{t-1}) \approx g(u_t, \mu_{t-1}) + \frac{\partial g(u_t, \mu_{t-1})}{\partial x_{t-1}} (x_{t-1} - \mu_{t-1})$$

$$g(u_t, x_{t-1}) \approx g(u_t, \mu_{t-1}) + G_t (x_{t-1} - \mu_{t-1})$$

- **Correction:**

$$h(x_t) \approx h(\bar{\mu}_t) + \frac{\partial h(\bar{\mu}_t)}{\partial x_t} (x_t - \bar{\mu}_t)$$

$$h(x_t) \approx h(\bar{\mu}_t) + H_t (x_t - \bar{\mu}_t)$$

Considering the Bayes filter theorem and the above approximation, an algorithm was created to find out the next state of a bot according to EKF filter. This algorithm is used for both localisation and mapping

1. **Extended_Kalman_filter**($\mu_{t-1}, \Sigma_{t-1}, u_t, z_t$):
2. Prediction:
3. $\bar{\mu}_t = g(u_t, \mu_{t-1})$ $\longleftarrow$ $\bar{\mu}_t = A_t \mu_{t-1} + B_t u_t$
4. $\bar{\Sigma}_t = G_t \Sigma_{t-1} G_t^T + R_t$ $\longleftarrow$ $\bar{\Sigma}_t = A_t \Sigma_{t-1} A_t^T + R_t$

5. Correction:

6. $K_t = \bar{\Sigma}_t H_t^T (H_t \bar{\Sigma}_t H_t^T + Q_t)^{-1}$ $\longleftarrow$ $K_t = \bar{\Sigma}_t C_t^T (C_t \bar{\Sigma}_t C_t^T + Q_t)^{-1}$

7. $\mu_t = \bar{\mu}_t + K_t(z_t - h(\bar{\mu}_t))$ $\longleftarrow$ $\mu_t = \bar{\mu}_t + K_t(z_t - C_t \bar{\mu}_t)$

8. $\Sigma_t = (I - K_t H_t)\bar{\Sigma}_t$ $\longleftarrow$ $\Sigma_t = (I - K_t C_t)\bar{\Sigma}_t$

9. Return μ_t, Σ_t

$$H_t = \frac{\partial h(\bar{\mu}_t)}{\partial x_t} \qquad G_t = \frac{\partial g(u_t, \mu_{t-1})}{\partial x_{t-1}}$$

The right-hand side of the above algorithm is for Kalman filters and the left-hand side is for Extended Kalman filters after considering the linear approximation.

The term K in the above algorithm is called as the Kalman gain of the filter, it is also called as the correction factor of the state process.

The above algorithm is used for localisation and map taking in the sensor data from a bot and using the data to solve the SLAM problem which is mostly considered as a Chicken-or-egg problem (which came first, the chicken or the egg).

From the data collected after the EKF, maps are drawn using various models. The most frequent one's is the grid model where the points where the landmarks are present are considered as 0 and the free spaces as 1 in a binary 2-D grid map.

The implementation of this so called algorithm to implement SLAM is shown in the simulation done in MATLAB in the next section.

SECTION II

SIMULATION IN MATLAB

Simulation 1:

To implement the EKF SLAM, we studied a MATLAB code and created a 2-D virtual map filled with landmarks and fixed waypoints on the path the bot is expected to follow and stored this map and the data on the map as matrices. Then we ran the code and inferred the results.

MATLAB code:

The code runs when the function ekfslam_sim () is called with the arguments from the data collected by creating the 2D virtual map filled with landmarks and waypoints. This opens a figure containing the map and the simulation of the bot takes place in this map. At the end of the simulation a structure of data is given as output which gives information about the no of states and its true and estimated position throughout these states and the diagonals of the covariance matrix. The estimated position is the one calculated according to the EKF slam algorithm after adding noise to the true path.

Code to build the 2D virtual map

```
function varargout = frontend(varargin)
%EKF-SLAM environment-making GUI
%
% This program permits the graphical creation and manipulation
% of an environment of point landmarks, and the specification of
% vehicle path waypoints therein.
%
% USAGE: type 'frontend' to start.
%   1. Click on the desired operation: <enter>, <move>, or <delete>.
%   2. Click on the type: <waypoint> or <landmark> to commence the
%   operation.
%   3. If entering new landmarks or waypoints, click with the left
%   mouse button to add new points. Click the right mouse button, or
%   hit <enter> key to finish.
%   4. To move or delete a point, just click near the desired point.
%   5. Saving maps and loading previous maps is accomplished via the
%   <save> and <load> buttons, respectively.
%
% Tim Bailey and Juan Nieto 2004.

% FRONTEND Application M-file for frontend.fig
%    FIG = FRONTEND launch frontend GUI.
%    FRONTEND('callback_name', ...) invoke the named callback.
global WAYPOINTS LANDMARKS FH

if nargin == 0  % LAUNCH GUI

    %initialisation
    WAYPOINTS= [0;0];
    LANDMARKS= [];
```

```
    % open figure
    fig = openfig(mfilename,'reuse');
    hh= get(fig, 'children');
    set(hh(3), 'value', 1)

    hold on
    FH.hl= plot(0,0,'g*'); plot(0,0,'w*')
    FH.hw= plot(0,0,0,0,'ro');
    plotwaypoints(WAYPOINTS);

    % Use system color scheme for figure:
    set(fig,'Color',get(0,'defaultUicontrolBackgroundColor'));
    set(fig,'name', 'SLAM Map-Making GUI')

    % Generate a structure of handles to pass to callbacks, and store it.
    handles = guihandles(fig);
    guidata(fig, handles);

    if nargout > 0
        varargout{1} = fig;
    end

elseif ischar(varargin{1}) % INVOKE NAMED SUBFUNCTION OR CALLBACK

    try
        [varargout{1:nargout}] = feval(varargin{:}); % FEVAL switchyard
    catch
        disp(lasterr);
    end

end


% --------------------------------------------------------------------
function varargout = waypoint_checkbox_Callback(h, eventdata, handles,
varargin)
global WAYPOINTS
set(handles.landmark_checkbox, 'value', 0)
WAYPOINTS= perform_task(WAYPOINTS, handles.waypoint_checkbox, handles);
plotwaypoints(WAYPOINTS);

% --------------------------------------------------------------------
function varargout = landmark_checkbox_Callback(h, eventdata, handles,
varargin)
global LANDMARKS
set(handles.waypoint_checkbox, 'value', 0)
LANDMARKS= perform_task(LANDMARKS, handles.landmark_checkbox, handles);
plotlandmarks(LANDMARKS);

% --------------------------------------------------------------------
function varargout = enter_checkbox_Callback(h, eventdata, handles,
varargin)
set(handles.enter_checkbox, 'value', 1)
set(handles.move_checkbox, 'value', 0)
set(handles.delete_checkbox, 'value', 0)

% --------------------------------------------------------------------
```

```
function varargout = move_checkbox_Callback(h, eventdata, handles,
varargin)
set(handles.enter_checkbox, 'value', 0)
set(handles.move_checkbox, 'value', 1)
set(handles.delete_checkbox, 'value', 0)

% --------------------------------------------------------------------
function varargout = delete_checkbox_Callback(h, eventdata, handles,
varargin)
set(handles.enter_checkbox, 'value', 0)
set(handles.move_checkbox, 'value', 0)
set(handles.delete_checkbox, 'value', 1)

% --------------------------------------------------------------------
function varargout = load_button_Callback(h, eventdata, handles, varargin)
global WAYPOINTS LANDMARKS
seed = {'*.mat','MAT-files (*.mat)'};
[fn,pn] = uigetfile(seed, 'Load landmarks and waypoints');
if fn==0, return, end

fnpn = strrep(fullfile(pn,fn), '''', '''''');
load(fnpn)
WAYPOINTS= wp; LANDMARKS= lm;
plotwaypoints(WAYPOINTS);
plotlandmarks(LANDMARKS);

% --------------------------------------------------------------------
function varargout = save_button_Callback(h, eventdata, handles, varargin)
global WAYPOINTS LANDMARKS
wp= WAYPOINTS; lm= LANDMARKS;
seed = {'*.mat','MAT-files (*.mat)'};
[fn,pn] = uiputfile(seed, 'Save landmarks and waypoints');
if fn==0, return, end

fnpn = strrep(fullfile(pn,fn), '''', '''''');
save(fnpn, 'wp', 'lm');

% --------------------------------------------------------------------
function plotwaypoints(x)
global FH
set(FH.hw(1), 'xdata', x(1,:), 'ydata', x(2,:))
set(FH.hw(2), 'xdata', x(1,:), 'ydata', x(2,:))

% --------------------------------------------------------------------
function plotlandmarks(x)
global FH
set(FH.hl, 'xdata', x(1,:), 'ydata', x(2,:))

% --------------------------------------------------------------------
function i= find_nearest(x)
xp= ginput(1);
d2= (x(1,:)-xp(1)).^2 + (x(2,:)-xp(2)).^2;
i= find(d2 == min(d2));
i= i(1);

% --------------------------------------------------------------------
function x= perform_task(x, h, handles)

if get(h, 'value') == 1
```

```
    zoom off

    if get(handles.enter_checkbox, 'value') == 1 % enter points
        [xn,yn,bn]= ginput(1);
        while ~isempty(xn) & bn == 1
            x= [x [xn;yn]];
            if h == handles.waypoint_checkbox
                plotwaypoints(x);
            else
                plotlandmarks(x);
            end
            [xn,yn,bn]= ginput(1);
        end
    else
        i= find_nearest(x);
        if get(handles.delete_checkbox, 'value') == 1 % delete nearest
point
            x= [x(:,1:i-1) x(:,i+1:end)];

        elseif get(handles.move_checkbox, 'value') == 1 % move nearest
point
            xt= x(:,i);
            plot(xt(1), xt(2),'kx', 'markersize',10)
            x(:,i)= ginput(1)';
            plot(xt(1), xt(2),'wx', 'markersize',10)
        end
    end

    set(h, 'value', 0)
end
```

Configuration file (To set the required control values)

```
%%% Configuration file
%%% Permits various adjustments to parameters of the SLAM algorithm.
%%% See ekfslam_sim.m for more information

% control parameters
V= 3; % m/s
MAXG= 30*pi/180; % radians, maximum steering angle (-MAXG < g < MAXG)
RATEG= 20*pi/180; % rad/s, maximum rate of change in steer angle
WHEELBASE= 4; % metres, vehicle wheel-base
DT_CONTROLS= 0.025; % seconds, time interval between control signals

% control noises
sigmaV= 0.3; % m/s
sigmaG= (3.0*pi/180); % radians
Q= [sigmaV^2 0; 0 sigmaG^2];

% observation parameters
MAX_RANGE= 30.0; % metres
DT_OBSERVE= 8*DT_CONTROLS; % seconds, time interval between observations

% observation noises
sigmaR= 0.1; % metres
sigmaB= (1.0*pi/180); % radians
R= [sigmaR^2 0; 0 sigmaB^2];
```

```
% data association innovation gates (Mahalanobis distances)
GATE_REJECT= 4.0; % maximum distance for association
GATE_AUGMENT= 25.0; % minimum distance for creation of new feature
% For 2-D observation:
%   - common gates are: 1-sigma (1.0), 2-sigma (4.0), 3-sigma (9.0), 4-
sigma (16.0)
%   - percent probability mass is: 1-sigma bounds 40%, 2-sigma 86%, 3-sigma
99%, 4-sigma 99.9%.

% waypoint proximity
AT_WAYPOINT= 1.0; % metres, distance from current waypoint at which to
switch to next waypoint
NUMBER_LOOPS= 2; % number of loops through the waypoint list

% switches
SWITCH_CONTROL_NOISE= 1; % if 0, velocity and gamma are perfect
SWITCH_SENSOR_NOISE = 1; % if 0, measurements are perfect
SWITCH_INFLATE_NOISE= 0; % if 1, the estimated Q and R are inflated (ie,
add stabilising noise)
SWITCH_HEADING_KNOWN= 0; % if 1, the vehicle heading is observed directly
at each iteration
SWITCH_ASSOCIATION_KNOWN= 0; % if 1, associations are given, if 0, they are
estimated using gates
SWITCH_BATCH_UPDATE= 1; % if 1, process scan in batch, if 0, process
sequentially
SWITCH_SEED_RANDOM= 0; % if not 0, seed the randn() with its value at
beginning of simulation (for repeatability)
SWITCH_USE_IEKF= 0; % if 1, use iterated EKF for updates, if 0, use normal
EKF
```

Function ekfslam_sim (main function to be called)

```
function data= ekfslam_sim(lm, wp)
format compact
configfile; % ** USE THIS FILE TO CONFIGURE THE EKF-SLAM **

% setup plots
fig=figure;
plot(lm(1,:),lm(2,:),'b*')
hold on, axis equal
plot(wp(1,:),wp(2,:), 'g', wp(1,:),wp(2,:),'g.')
xlabel('metres'), ylabel('metres')
set(fig, 'name', 'EKF-SLAM Simulator')
h= setup_animations;
veh= [0 -WHEELBASE -WHEELBASE; 0 -2 2]; % vehicle animation
plines=[]; % for laser line animation
pcount=0;

% initialise states
xtrue= zeros(3,1);
x= zeros(3,1);
P= zeros(3);

% initialise other variables and constants
dt= DT_CONTROLS; % change in time between predicts
dtsum= 0; % change in time since last observation
ftag= 1:size(lm,2); % identifier for each landmark
da_table= zeros(1,size(lm,2)); % data association table
```

```
iwp= 1; % index to first waypoint
G= 0; % initial steer angle
data= initialise_store(x,P,x); % stored data for off-line
QE= Q; RE= R; if SWITCH_INFLATE_NOISE, QE= 2*Q; RE= 8*R; end % inflate
estimated noises (ie, add stabilising noise)
if SWITCH_SEED_RANDOM, randn('state',SWITCH_SEED_RANDOM), end

% main loop
while iwp ~= 0

    % compute true data
    [G,iwp]= compute_steering(xtrue, wp, iwp, AT_WAYPOINT, G, RATEG, MAXG,
dt);
    if iwp==0 & NUMBER_LOOPS > 1, iwp=1; NUMBER_LOOPS= NUMBER_LOOPS-1; end
% perform loops: if final waypoint reached, go back to first
    xtrue= vehicle_model(xtrue, V,G, WHEELBASE,dt);
    [Vn,Gn]= add_control_noise(V,G,Q, SWITCH_CONTROL_NOISE);

    % EKF predict step
    [x,P]= predict (x,P, Vn,Gn,QE, WHEELBASE,dt);

    % if heading known, observe heading
    [x,P]= observe_heading(x,P, xtrue(3), SWITCH_HEADING_KNOWN);

    % EKF update step
    dtsum= dtsum + dt;
    if dtsum >= DT_OBSERVE
        dtsum= 0;
        [z,ftag_visible]= get_observations(xtrue, lm, ftag, MAX_RANGE);
        z= add_observation_noise(z,R, SWITCH_SENSOR_NOISE);

        if SWITCH_ASSOCIATION_KNOWN == 1
            [zf,idf,zn, da_table]= data_associate_known(x,z,ftag_visible,
da_table);
        else
            [zf,idf, zn]= data_associate(x,P,z,RE, GATE_REJECT,
GATE_AUGMENT);
        end

        if SWITCH_USE_IEKF == 1
            [x,P]= update_iekf(x,P,zf,RE,idf, 5);
        else
            [x,P]= update(x,P,zf,RE,idf, SWITCH_BATCH_UPDATE);
        end
        [x,P]= augment(x,P, zn,RE);
    end

    % offline data store
    data= store_data(data, x, P, xtrue);

    % plots
    xt= TransformToGlobal(veh,xtrue);
    xv= TransformToGlobal(veh,x(1:3));
    set(h.xt, 'xdata', xt(1,:), 'ydata', xt(2,:))
    set(h.xv, 'xdata', xv(1,:), 'ydata', xv(2,:))
    set(h.xf, 'xdata', x(4:2:end), 'ydata', x(5:2:end))
    ptmp= make_covariance_ellipses(x(1:3),P(1:3,1:3));
    pcov(:,1:size(ptmp,2))= ptmp;
    if dtsum==0
```

```
        set(h.cov, 'xdata', pcov(1,:), 'ydata', pcov(2,:))
        pcount= pcount+1;
        if pcount == 15
            set(h.pth, 'xdata', data.path(1,1:data.i), 'ydata',
data.path(2,1:data.i))
            pcount=0;
        end
        if ~isempty(z)
            plines= make_laser_lines (z,x(1:3));
            set(h.obs, 'xdata', plines(1,:), 'ydata', plines(2,:))
            pcov= make_covariance_ellipses(x,P);
        end
    end
    drawnow
end

data= finalise_data(data);
set(h.pth, 'xdata', data.path(1,:), 'ydata', data.path(2,:))

%
%

function h= setup_animations()
h.xt= patch(0,0,'b','erasemode','xor'); % vehicle true
h.xv= patch(0,0,'r','erasemode','xor'); % vehicle estimate
h.pth= plot(0,0,'k.','markersize',2,'erasemode','background'); % vehicle
path estimate
h.obs= plot(0,0,'r','erasemode','xor'); % observations
h.xf= plot(0,0,'r+','erasemode','xor'); % estimated features
h.cov= plot(0,0,'r','erasemode','xor'); % covariance ellipses

%
%

function p= make_laser_lines (rb,xv)
% compute set of line segments for laser range-bearing measurements
if isempty(rb), p=[]; return, end
len= size(rb,2);
lnes(1,:)= zeros(1,len)+ xv(1);
lnes(2,:)= zeros(1,len)+ xv(2);
lnes(3:4,:)= TransformToGlobal([rb(1,:).*cos(rb(2,:));
rb(1,:).*sin(rb(2,:))], xv);
p= line_plot_conversion (lnes);

%
%

function p= make_covariance_ellipses(x,P)
% compute ellipses for plotting state covariances
N= 10;
inc= 2*pi/N;
phi= 0:inc:2*pi;

lenx= length(x);
lenf= (lenx-3)/2;
p= zeros (2,(lenf+1)*(N+2));

ii=1:N+2;
p(:,ii)= make_ellipse(x(1:2), P(1:2,1:2), 2, phi);
```

```
ctr= N+3;
for i=1:lenf
    ii= ctr:(ctr+N+1);
    jj= 2+2*i; jj= jj:jj+1;

    p(:,ii)= make_ellipse(x(jj), P(jj,jj), 2, phi);
    ctr= ctr+N+2;
end

%
%

function p= make_ellipse(x,P,s, phi)
% make a single 2-D ellipse of s-sigmas over phi angle intervals
r= sqrtm(P);
a= s*r*[cos(phi); sin(phi)];
p(2,:)= [a(2,:)+x(2) NaN];
p(1,:)= [a(1,:)+x(1) NaN];

%
%

function data= initialise_store(x,P, xtrue)
% offline storage initialisation
data.i=1;
data.path= x;
data.true= xtrue;
data.state(1).x= x;
%data.state(1).P= P;
data.state(1).P= diag(P);

%
%

function data= store_data(data, x, P, xtrue)
% add current data to offline storage
CHUNK= 5000;
if data.i == size(data.path,2) % grow array in chunks to amortise
reallocation
    data.path= [data.path zeros(3,CHUNK)];
    data.true= [data.true zeros(3,CHUNK)];
end
i= data.i + 1;
data.i= i;
data.path(:,i)= x(1:3);
data.true(:,i)= xtrue;
data.state(i).x= x;
%data.state(i).P= P;
data.state(i).P= diag(P);

%
%

function data= finalise_data(data)
% offline storage finalisation
data.path= data.path(:,1:data.i);
data.true= data.true(:,1:data.i);
```

Function compute_steering

```
function [G,iwp]= compute_steering(x, wp, iwp, minD, G, rateG, maxG, dt)
%function [G,iwp]= compute_steering(x, wp, iwp, minD, G, rateG, maxG, dt)
%
% INPUTS:
%   x - true position
%   wp - waypoints
%   iwp - index to current waypoint
%   minD - minimum distance to current waypoint before switching to next
%   G - current steering angle
%   rateG - max steering rate (rad/s)
%   maxG - max steering angle (rad)
%   dt - timestep
%
% OUTPUTS:
%   G - new current steering angle
%   iwp - new current waypoint
%


% determine if current waypoint reached
cwp= wp(:,iwp);
d2= (cwp(1)-x(1))^2 + (cwp(2)-x(2))^2;
if d2 < minD^2
    iwp= iwp+1; % switch to next
    if iwp > size(wp,2) % reached final waypoint, flag and return
        iwp=0;
        return;
    end
    cwp= wp(:,iwp); % next waypoint
end

% compute change in G to point towards current waypoint
deltaG= pi_to_pi(atan2(cwp(2)-x(2), cwp(1)-x(1)) - x(3) - G);

% limit rate
maxDelta= rateG*dt;
if abs(deltaG) > maxDelta
    deltaG= sign(deltaG)*maxDelta;
end

% limit angle
G= G+deltaG;
if abs(G) > maxG
    G= sign(G)*maxG;
end
```

Function add_control_noise

```
function [V,G]= add_control_noise(V,G,Q, addnoise)
%function [V,G]= add_control_noise(V,G,Q, addnoise)
%
% Add random noise to nominal control values. We assume Q is diagonal.
if addnoise == 1
```

```
    V= V + randn(1)*sqrt(Q(1,1));
    G= G + randn(1)*sqrt(Q(2,2));
end
```

Function add_observation_noise

```
function z= add_observation_noise(z,R, addnoise)
%function z= add_observation_noise(z,R, addnoise)
%
% Add random measurement noise. We assume R is diagonal.

if addnoise == 1
    len= size(z,2);
    if len > 0
        z(1,:)= z(1,:) + randn(1,len)*sqrt(R(1,1));
        z(2,:)= z(2,:) + randn(1,len)*sqrt(R(2,2));
    end
end
```

Function augment

```
function [x,P]= augment(x,P,z,R)
%function [x,P]= augment(x,P,z,R)
%
% Inputs:
%   x, P - SLAM state and covariance
%   z, R - range-bearing measurements and covariances, each of a new
feature
%
% Outputs:
%   x, P - augmented state and covariance
%
% Notes:
%   - We assume the number of vehicle pose states is three.
%   - Only one value for R is used, as all measurements are assumed to
%   have same noise properties.
%
% Tim Bailey 2004.

% add new features to state
for i=1:size(z,2)
    [x,P]= add_one_z(x,P,z(:,i),R);
end

%
%

function [x,P]= add_one_z(x,P,z,R)

len= length(x);
r= z(1); b= z(2);
s= sin(x(3)+b);
c= cos(x(3)+b);

% augment x
x= [x;
    x(1) + r*c;
```

```
    x(2) + r*s];

% jacobians
Gv= [1 0 -r*s;
     0 1  r*c];
Gz= [c -r*s;
     s  r*c];

% augment P
rng= len+1:len+2;
P(rng,rng)= Gv*P(1:3,1:3)*Gv' + Gz*R*Gz'; % feature cov
P(rng,1:3)= Gv*P(1:3,1:3); % vehicle to feature xcorr
P(1:3,rng)= P(rng,1:3)';
if len>3
    rnm= 4:len;
    P(rng,rnm)= Gv*P(1:3,rnm); % map to feature xcorr
    P(rnm,rng)= P(rng,rnm)';
end
```

Function data_associate

```
function [zf,idf, zn]= data_associate(x,P,z,R, gate1, gate2)
%
% Simple gated nearest-neighbour data-association. No clever feature
% caching tricks to speed up association, so computation is O(N), where
% N is the number of features in the state.
%
% Tim Bailey 2004.

zf= []; zn= [];
idf= [];

Nxv= 3; % number of vehicle pose states
Nf= (length(x) - Nxv)/2; % number of features already in map

% linear search for nearest-neighbour, no clever tricks (like a quick
% bounding-box threshold to remove distant features; or, better yet,
% a balanced k-d tree lookup). TODO: implement clever tricks.
for i=1:size(z,2)
    jbest= 0;
    nbest= inf;
    outer= inf;

    % search for neighbours
    for j=1:Nf
        [nis, nd]= compute_association(x,P,z(:,i),R, j);
        if nis < gate1 & nd < nbest % if within gate, store nearest-
neighbour
            nbest= nd;
            jbest= j;
        elseif nis < outer % else store best nis value
            outer= nis;
        end
    end

    % add nearest-neighbour to association list
    if jbest ~= 0
```

```
        zf=  [zf  z(:,i)];
        idf= [idf jbest];
    elseif outer > gate2 % z too far to associate, but far enough to be a
new feature
        zn= [zn z(:,i)];
    end
end

function [nis, nd]= compute_association(x,P,z,R,idf)
%
% return normalised innovation squared (ie, Mahalanobis distance) and
normalised distance
[zp,H]= observe_model(x, idf);
v= z-zp;
v(2)= pi_to_pi(v(2));
S= H*P*H' + R;

nis= v'*inv(S)*v;
nd= nis + log(det(S));
```

Function get_observations

```
 function [z,idf]= get_observations(x, lm, idf, rmax)
%function [z,idf]= get_observations(x, lm, idf, rmax)
%
% INPUTS:
%   x - vehicle pose [x;y;phi]
%   lm - set of all landmarks
%   idf - index tags for each landmark
%   rmax - maximum range of range-bearing sensor
%
% OUTPUTS:
%   z - set of range-bearing observations
%   idf - landmark index tag for each observation
%
% Tim Bailey 2004.

[lm,idf]= get_visible_landmarks(x,lm,idf,rmax);
z= compute_range_bearing(x,lm);

%
%

function [lm,idf]= get_visible_landmarks(x,lm,idf,rmax)
% Select set of landmarks that are visible within vehicle's semi-circular
field-of-view
dx= lm(1,:) - x(1);
dy= lm(2,:) - x(2);
phi= x(3);

% incremental tests for bounding semi-circle
ii= find(abs(dx) < rmax & abs(dy) < rmax ... % bounding box
      & (dx*cos(phi) + dy*sin(phi)) > 0 ...  % bounding line
      & (dx.^2 + dy.^2) < rmax^2);           % bounding circle
% Note: the bounding box test is unnecessary but illustrates a possible
speedup technique
% as it quickly eliminates distant points. Ordering the landmark set would
make this operation
% O(logN) rather that O(N).
```

```
lm= lm(:,ii);
idf= idf(ii);

%
%

function z= compute_range_bearing(x,lm)
% Compute exact observation
dx= lm(1,:) - x(1);
dy= lm(2,:) - x(2);
phi= x(3);
z= [sqrt(dx.^2 + dy.^2);
    atan2(dy,dx) - phi];
```

Function simple_update

```
function [z,idf]= get_observations(x, lm, idf, rmax)
%function [z,idf]= get_observations(x, lm, idf, rmax)
%
% INPUTS:
%   x - vehicle pose [x;y;phi]
%   lm - set of all landmarks
%   idf - index tags for each landmark
%   rmax - maximum range of range-bearing sensor
%
% OUTPUTS:
%   z - set of range-bearing observations
%   idf - landmark index tag for each observation
%
% Tim Bailey 2004.

[lm,idf]= get_visible_landmarks(x,lm,idf,rmax);
z= compute_range_bearing(x,lm);

%
%

function [lm,idf]= get_visible_landmarks(x,lm,idf,rmax)
% Select set of landmarks that are visible within vehicle's semi-circular
field-of-view
dx= lm(1,:) - x(1);
dy= lm(2,:) - x(2);
phi= x(3);

% incremental tests for bounding semi-circle
ii= find(abs(dx) < rmax & abs(dy) < rmax ... % bounding box
      & (dx*cos(phi) + dy*sin(phi)) > 0 ...  % bounding line
      & (dx.^2 + dy.^2) < rmax^2);           % bounding circle
% Note: the bounding box test is unnecessary but illustrates a possible
speedup technique
% as it quickly eliminates distant points. Ordering the landmark set would
make this operation
% O(logN) rather that O(N).

lm= lm(:,ii);
idf= idf(ii);
```

```
%
%

function z= compute_range_bearing(x,lm)
% Compute exact observation
dx= lm(1,:) - x(1);
dy= lm(2,:) - x(2);
phi= x(3);
z= [sqrt(dx.^2 + dy.^2);
    atan2(dy,dx) - phi];
```

Function line_plot_conversion

```
function p= line_plot_conversion (lne)
%function p= line_plot_conversion (lne)
%
% INPUT: list of lines [x1;y1;x2;y2]
% OUTPUT: list of points [x;y]
%
% Convert a list of lines so that they will be plotted as a set of
% unconnected lines but only require a single handle to do so. This
% is performed by converting the lines to a set of points, where a
% NaN point is inserted between every point-pair:
%
%   l= [x1a x1b x1c;
%       y1a y1b y1c;
%       x2a x2b x2c;
%       y2a y2b y2c];
%
%   becomes
%
%   p= [x1a x2a NaN x1b x2b NaN x1c x2c;
%       y1a y2a NaN y1b y2b NaN y1c y2c];
%
% Tim Bailey 2002. Thanks to Jose Guivant for this 'discrete lines'
% plotting technique.

len= size(lne,2)*3 - 1;
p= zeros(2, len);

p(:,1:3:end)= lne(1:2,:);
p(:,2:3:end)= lne(3:4,:);
p(:,3:3:end)= NaN;
```

Function observe_model

```
function [z,H]= observe_model(x, idf)
%function [z,H]= observe_model(x, idf)
%
% INPUTS:
%   x - state vector
%   idf - index of feature order in state
%
% OUTPUTS:
%   z - predicted observation
```

```
%   H - observation Jacobian
%
% Given a feature index (ie, the order of the feature in the state vector),
% predict the expected range-bearing observation of this feature and its
Jacobian.
%
% Tim Bailey 2004.

Nxv= 3; % number of vehicle pose states
fpos= Nxv + idf*2 - 1; % position of xf in state
H= zeros(2, length(x));

% auxiliary values
dx= x(fpos)  -x(1);
dy= x(fpos+1)-x(2);
d2= dx^2 + dy^2;
d= sqrt(d2);
xd= dx/d;
yd= dy/d;
xd2= dx/d2;
yd2= dy/d2;

% predict z
z= [d;
    atan2(dy,dx) - x(3)];

% calculate H
H(:,1:3)        = [-xd -yd 0; yd2 -xd2 -1];
H(:,fpos:fpos+1)= [ xd  yd;  -yd2  xd2];
```

Function pi_to_pi

```
function angle = pi_to_pi(angle)

%function angle = pi_to_pi(angle)
% Input: array of angles.
% Tim Bailey 2000

angle = mod(angle, 2*pi);

i=find(angle>pi);
angle(i)=angle(i)-2*pi;

i=find(angle<-pi);
angle(i)=angle(i)+2*pi;
```

Function plot_feature_loci

```
function plot_feature_loci(data)
%function plot_feature_loci(data)
%
% From the offline stored data of the simulator, plot the
% paths of the feature estimates over time.
%

hold on
N= length(data.state);
```

```
maxf= length(data.state(N).x) - 3;
flocus= zeros(maxf,N);
iold= maxf;

for i = N:-1:1
    ii= length(data.state(i).x)- 3;
    if ii>0, flocus(1:ii, i)= data.state(i).x(4:end); end

    if ii<iold
        for j=(ii+1):2:iold
            jj= (i+1):N;
            plot(flocus(j, jj), flocus(j+1, jj), 'k')
        end
        iold= ii;
    end
end
```

Function predict

```
function [x,P]= predict (x,P,v,g,Q,WB,dt)
%function [xn,Pn]= predict (x,P,v,g,Q,WB,dt)
%
% Inputs:
%   x, P - SLAM state and covariance
%   v, g - control inputs: velocity and gamma (steer angle)
%   Q - covariance matrix for velocity and gamma
%   WB - vehicle wheelbase
%   dt - timestep
%
% Outputs:
%   xn, Pn - predicted state and covariance
%
% Tim Bailey 2004.

s= sin(g+x(3)); c= cos(g+x(3));
vts= v*dt*s; vtc= v*dt*c;

% jacobians
Gv= [1 0 -vts;
     0 1  vtc;
     0 0 1];
Gu= [dt*c -vts;
     dt*s  vtc;
     dt*sin(g)/WB v*dt*cos(g)/WB];

% predict covariance
P(1:3,1:3)= Gv*P(1:3,1:3)*Gv' + Gu*Q*Gu';
if size(P,1)>3
    P(1:3,4:end)= Gv*P(1:3,4:end);
    P(4:end,1:3)= P(1:3,4:end)';
end

% predict state
x(1:3)= [x(1) + vtc;
         x(2) + vts;
         pi_to_pi(x(3)+ v*dt*sin(g)/WB)];
```

Function TransformToGlobal

```
function p = TransformToGlobal(p, b)
% function p = TransformToGlobal(p, b)
%
% Transform a list of poses [x;y;phi] so that they are global wrt a base
pose
%
% Tim Bailey 1999

% rotate
rot = [cos(b(3)) -sin(b(3)); sin(b(3)) cos(b(3))];
p(1:2,:) = rot*p(1:2,:);

% translate
p(1,:) = p(1,:) + b(1);
p(2,:) = p(2,:) + b(2);

% if p is a pose and not a point
if size(p,1)==3
   p(3,:) = pi_to_pi(p(3,:) + b(3));
end
```

Function update

```
function [x,P]= update(x,P,z,R,idf, batch)
% function [x,P]= update(x,P,z,R,idf, batch)
%
% Inputs:
%   x, P - SLAM state and covariance
%   z, R - range-bearing measurements and covariances
%   idf - feature index for each z
%   batch - switch to specify whether to process measurements together or
sequentially
%
% Outputs:
%   x, P - updated state and covariance

if batch == 1
    [x,P]= batch_update(x,P,z,R,idf);
else
    [x,P]= single_update(x,P,z,R,idf);
end

%
%

function [x,P]= batch_update(x,P,z,R,idf)

lenz= size(z,2);
lenx= length(x);
H= zeros(2*lenz, lenx);
v= zeros(2*lenz, 1);
RR= zeros(2*lenz);

for i=1:lenz
    ii= 2*i + (-1:0);
```

```
    [zp,H(ii,:)]= observe_model(x, idf(i));

    v(ii)=      [z(1,i)-zp(1);
        pi_to_pi(z(2,i)-zp(2))];
    RR(ii,ii)= R;
end

[x,P]= KF_cholesky_update(x,P,v,RR,H);

%
%

function [x,P]= single_update(x,P,z,R,idf)

lenz= size(z,2);
for i=1:lenz
    [zp,H]= observe_model(x, idf(i));

    v= [z(1,i)-zp(1);
        pi_to_pi(z(2,i)-zp(2))];

    [x,P]= KF_cholesky_update(x,P,v,RR,H);
end
```

Function vehicle_model

```
function xv= vehicle_model(xv, V,G, WB,dt)
%
% INPUTS:
%   xv - vehicle pose [x;y;phi]
%   V - velocity
%   G - steer angle (gamma)
%   WB - wheelbase
%   dt - change in time
%
% OUTPUTS:
%   xv - new vehicle pose

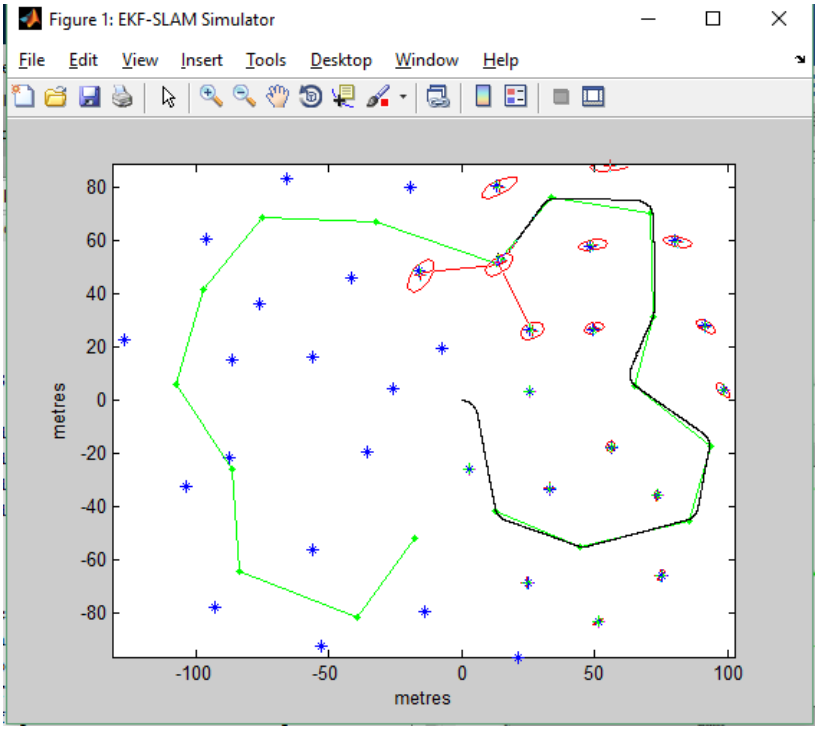
xv= [xv(1) + V*dt*cos(G+xv(3,:));
     xv(2) + V*dt*sin(G+xv(3,:));
     pi_to_pi(xv(3) + V*dt*sin(G)/WB)];
```

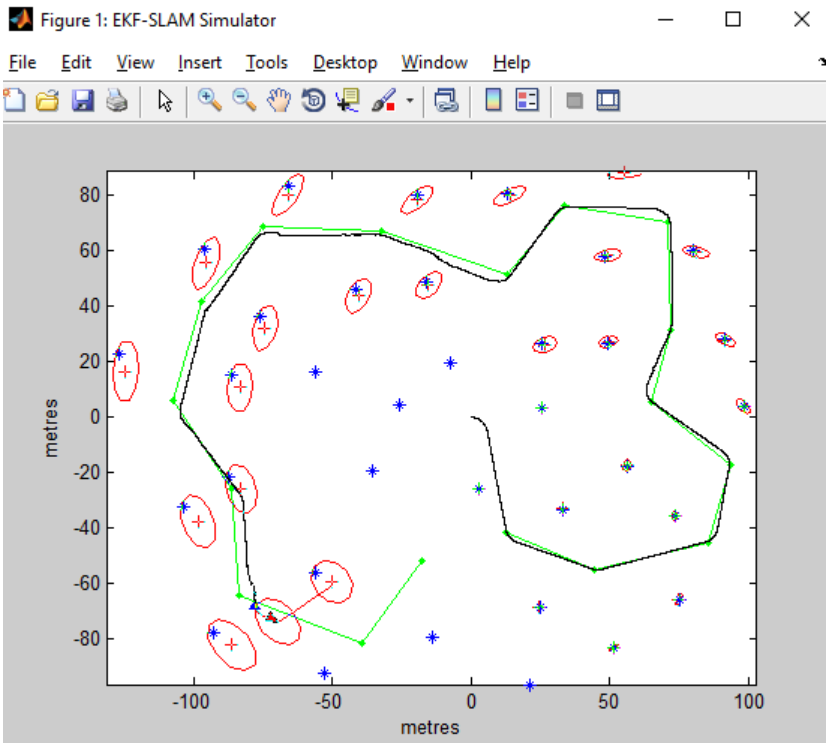
RESULT AND INFERENCE

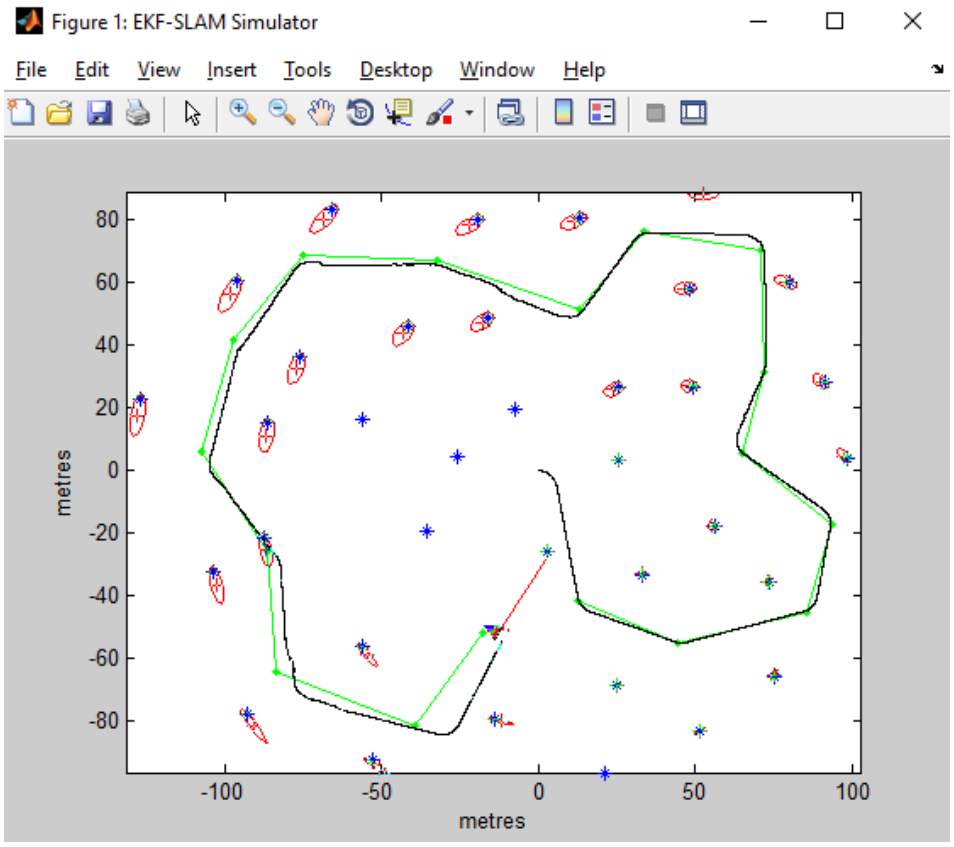
The simulation was run and it was observed that when the bot finished its first round across the waypoints the uncertainty of the landmarks was high whereas when it started its second round and went back to an already visited place, the uncertainty of the landmarks reduced as the bot had a chance to make its initial state a certain location to anchor the other landmarks to this place. This simulation showed the EKF estimation of the SLAM

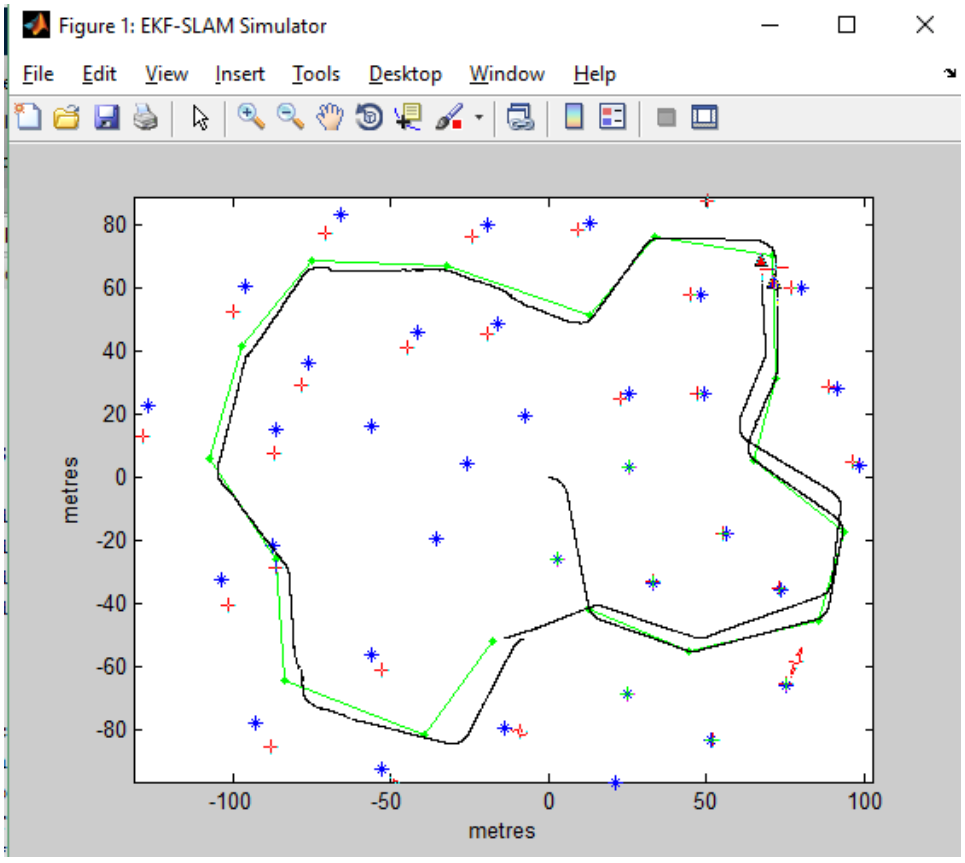
SNAPS FROM SIMULATION

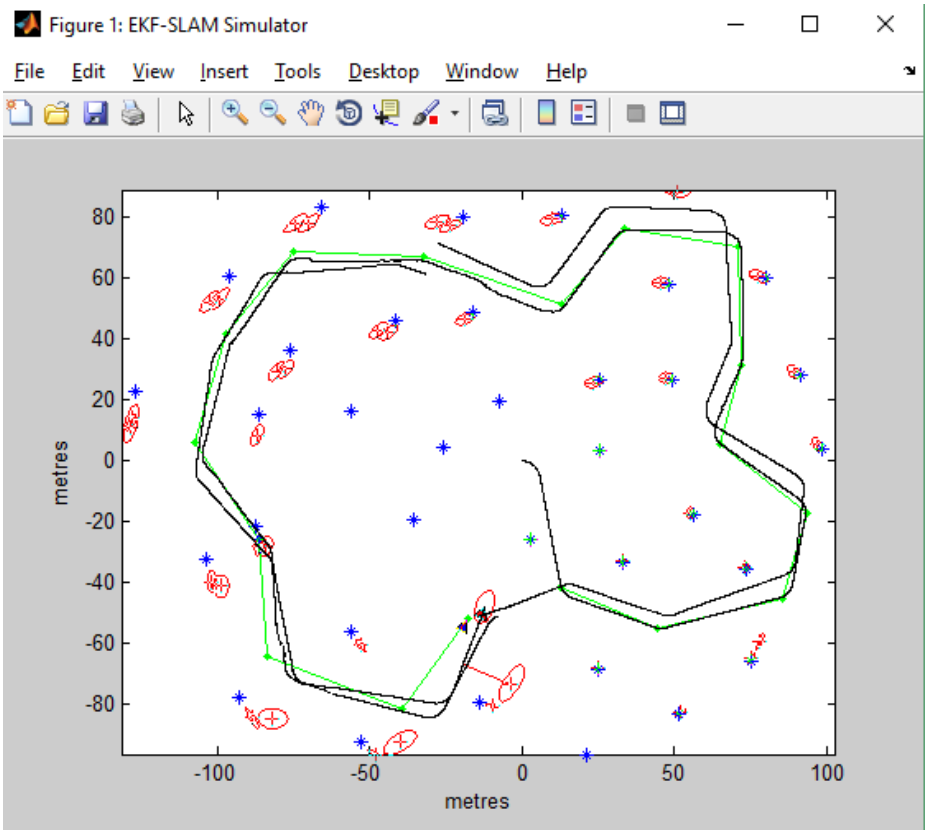
The Blue triangle is the true path of the bot.
The Red triangle is the predicted path of the bot.
The Blue points are the landmarks in the map.
The Green points are the waypoints
The Green line is the true path.






Figure 1: EKF-SLAM Simulator
File Edit View Insert Tools Desktop Window Help
80
60
40
20
0
-20
-40
-60
-80
metres
-100 -50 0 50 100
metres


Figure 1: EKF-SLAM Simulator
File Edit View Insert Tools Desktop Window Help
80
60
40
20
0
-20
-40
-60
-80
metres
-100 -50 0 50 100
metres



Simulation 2:

To further implement the EKF SLAM, we altered the previous MATLAB code and created a 2-D virtual map filled with landmarks alone and coded it such that the bot moves across this map such that it senses the landmarks and moves accordingly such that it doesn't collide with them.

MATLAB code:

Again, the code runs when the function ekfslam_sim () is called with the arguments from the data collected by creating the 2D virtual map filled with landmarks. This opens a figure containing the map and the simulation of the bot takes place in this map. At the end of the simulation a structure of data is given as output which gives information about the no of states and its true and estimated position throughout these states and the diagonals of the covariance matrix. The estimated position is the one calculated according to the EKF slam algorithm after adding noise to the true path.

The code was altered in some specific functions and the updated functions are given below. The Bot was also made to be circular and the sensor positions as three points in the edge of the bot.

Function ekfslam_sim

```
    function data= ekfslam_sim(lm, wp)
%function data= ekfslam_sim(lm, wp)
%
% INPUTS:
%   lm - set of landmarks
%   wp - set of waypoints
%
% OUTPUTS:
%   data - a data structure containing:
%       data.true: the vehicle 'true'-path (ie, where the vehicle
*actually* went)
%       data.path: the vehicle path estimate (ie, where SLAM estimates the
vehicle went)
%       data.state(k).x: the SLAM state vector at time k
%       data.state(k).P: the diagonals of the SLAM covariance matrix at
time k
%
% NOTES:
%   This program is a SLAM simulator. To use, create a set of landmarks and
%   vehicle waypoints (ie, waypoints for the desired vehicle path). The
program
%   'frontend.m' may be used to create this simulated environment - type
%   'help frontend' for more information.
%       The configuration of the simulator is managed by the script file
%   'configfile.m'. To alter the parameters of the vehicle, sensors, etc
%   adjust this file. There are also several switches that control certain
%   filter options.
%
% Tim Bailey and Juan Nieto 2004.
% Version 1.0


format compact
configfile; % ** USE THIS FILE TO CONFIGURE THE EKF-SLAM **

% setup plots
fig=figure;
plot(lm(1,:),lm(2,:),'b*')
hold on, axis equal
plot(wp(1,:),wp(2,:), 'g', wp(1,:),wp(2,:),'g.')
xlabel('metres'), ylabel('metres')
set(fig, 'name', 'EKF-SLAM Simulator')
h= setup_animations;
t = linspace(0, 2*pi);
x = WHEELBASE*cos(t);
y = WHEELBASE*sin(t);
veh= [x;y];% vehicle animation
le=length(t);
scanner=[veh(1,1),veh(1,round(le/6)),veh(1,round(10*le/12)+1);veh(2,1),veh(
2,round(le/6)),veh(2,round(10*le/12)+1)];
```

```
plines=[]; % for laser line animation
pcount=0;

% initialise states
xtrue= zeros(3,1);
x= zeros(3,1);
P= zeros(3);

% initialise other variables and constants
dt= DT_CONTROLS; % change in time between predicts
dtsum= 0; % change in time since last observation
ftag= 1:size(lm,2); % identifier for each landmark
da_table= zeros(1,size(lm,2)); % data association table
iwp= 1; % index to first waypoint
G= 0; % initial steer angle
data= initialise_store(x,P,x); % stored data for off-line
QE= Q; RE= R; if SWITCH_INFLATE_NOISE, QE= 2*Q; RE= 8*R; end % inflate
estimated noises (ie, add stabilising noise)
if SWITCH_SEED_RANDOM, randn('state',SWITCH_SEED_RANDOM), end

% main loop
while iwp ~= 0

        dtsum= 0;
        [xtrue,G,z,ftag_visible]= get_observations(xtrue, lm, ftag,
MAX_RANGE,G,dt);
        [Vn,Gn]= add_control_noise(V,G,Q, SWITCH_CONTROL_NOISE);

    % EKF predict step
    [x,P]= predict (x,P, Vn,Gn,QE, WHEELBASE,dt);

    % if heading known, observe heading
    [x,P]= observe_heading(x,P, xtrue(3), SWITCH_HEADING_KNOWN);
        z= add_observation_noise(z,R, SWITCH_SENSOR_NOISE);

        if SWITCH_ASSOCIATION_KNOWN == 1
            [zf,idf,zn, da_table]= data_associate_known(x,z,ftag_visible,
da_table);
        else
            [zf,idf, zn]= data_associate(x,P,z,RE, GATE_REJECT,
GATE_AUGMENT);
        end

        if SWITCH_USE_IEKF == 1
            [x,P]= update_iekf(x,P,zf,RE,idf, 5);
        else
            [x,P]= update(x,P,zf,RE,idf, SWITCH_BATCH_UPDATE);
        end
        [x,P]= augment(x,P, zn,RE);


    % offline data store
    data= store_data(data, x, P, xtrue);

    % plots
    xt= TransformToGlobal(veh,xtrue);
    xv= TransformToGlobal(veh,x(1:3));
    st= TransformToGlobal(scanner,xtrue);
    sv= TransformToGlobal(scanner,x(1:3));
```

```
    set(h.xt, 'xdata', xt(1,:), 'ydata', xt(2,:))
    set(h.xv, 'xdata', xv(1,:), 'ydata', xv(2,:))
    set(h.st, 'xdata', st(1,:), 'ydata', st(2,:))
    set(h.sv, 'xdata', sv(1,:), 'ydata', sv(2,:))
    set(h.xf, 'xdata', x(4:2:end), 'ydata', x(5:2:end))
    ptmp= make_covariance_ellipses(x(1:3),P(1:3,1:3));
    pcov(:,1:size(ptmp,2))= ptmp;
    if dtsum==0
        set(h.cov, 'xdata', pcov(1,:), 'ydata', pcov(2,:))
        pcount= pcount+1;
        if pcount == 15
            set(h.pth, 'xdata', data.path(1,1:data.i), 'ydata',
data.path(2,1:data.i))
            pcount=0;
        end
        if ~isempty(z)
            plines= make_laser_lines (z,x(1:3));
            set(h.obs, 'xdata', plines(1,:), 'ydata', plines(2,:))
            pcov= make_covariance_ellipses(x,P);
        end
    end
    drawnow
end

data= finalise_data(data);
set(h.pth, 'xdata', data.path(1,:), 'ydata', data.path(2,:))

%
%

function h= setup_animations()
h.xt= patch(0,0,'b','erasemode','xor'); % vehicle true
h.xv= patch(0,0,'r','erasemode','xor'); % vehicle estimate
h.st= plot(zeros(1,3),zeros(1,3),'k*','erasemode','xor');
h.sv= plot(zeros(1,3),zeros(1,3),'k*','erasemode','xor');
h.pth= plot(0,0,'k.','markersize',2,'erasemode','background'); % vehicle
path estimate
h.obs= plot(0,0,'r','erasemode','xor'); % observations
h.xf= plot(0,0,'r+','erasemode','xor'); % estimated features
h.cov= plot(0,0,'r','erasemode','xor'); % covariance ellipses

%
%

function p= make_laser_lines (rb,xv)
% compute set of line segments for laser range-bearing measurements
if isempty(rb), p=[]; return, end
len= size(rb,2);
lnes(1,:)= zeros(1,len)+ xv(1);
lnes(2,:)= zeros(1,len)+ xv(2);
lnes(3:4,:)= TransformToGlobal([rb(1,:).*cos(rb(2,:));
rb(1,:).*sin(rb(2,:))], xv);
p= line_plot_conversion (lnes);

%
%

function p= make_covariance_ellipses(x,P)
% compute ellipses for plotting state covariances
N= 10;
```

```
inc= 2*pi/N;
phi= 0:inc:2*pi;

lenx= length(x);
lenf= (lenx-3)/2;
p= zeros (2,(lenf+1)*(N+2));

ii=1:N+2;
p(:,ii)= make_ellipse(x(1:2), P(1:2,1:2), 2, phi);

ctr= N+3;
for i=1:lenf
    ii= ctr:(ctr+N+1);
    jj= 2+2*i; jj= jj:jj+1;

    p(:,ii)= make_ellipse(x(jj), P(jj,jj), 2, phi);
    ctr= ctr+N+2;
end

%
%

function p= make_ellipse(x,P,s, phi)
% make a single 2-D ellipse of s-sigmas over phi angle intervals
r= sqrtm(P);
a= s*r*[cos(phi); sin(phi)];
p(2,:)= [a(2,:)+x(2) NaN];
p(1,:)= [a(1,:)+x(1) NaN];

%
%

function data= initialise_store(x,P, xtrue)
% offline storage initialisation
data.i=1;
data.path= x;
data.true= xtrue;
data.state(1).x= x;
%data.state(1).P= P;
data.state(1).P= diag(P);

%
%

function data= store_data(data, x, P, xtrue)
% add current data to offline storage
CHUNK= 5000;
if data.i == size(data.path,2) % grow array in chunks to amortise
reallocation
    data.path= [data.path zeros(3,CHUNK)];
    data.true= [data.true zeros(3,CHUNK)];
end
i= data.i + 1;
data.i= i;
data.path(:,i)= x(1:3);
data.true(:,i)= xtrue;
data.state(i).x= x;
%data.state(i).P= P;
data.state(i).P= diag(P);
```

```
%
%

function data= finalise_data(data)
% offline storage finalisation
data.path= data.path(:,1:data.i);
data.true= data.true(:,1:data.i);
```

Function get_observations

```
function [x,g,z,idf]= get_observations(x, lm, idf, rmax,g,dt)
%function [z,idf]= get_observations(x, lm, idf, rmax)
%
% INPUTS:
%   x - vehicle pose [x;y;phi]
%   lm - set of all landmarks
%   idf - index tags for each landmark
%   rmax - maximum range of range-bearing sensor
%
% OUTPUTS:
%   z - set of range-bearing observations
%   idf - landmark index tag for each observation
%
% Tim Bailey 2004.
[x,g,lm,idf]= get_visible_landmarks(x,lm,idf,rmax,g,dt);
z= compute_range_bearing(x,lm);

%
%

function [x,g,lm,idf]= get_visible_landmarks(x,lm,idf,rmax,g,dt)
% Select set of landmarks that are visible within vehicle's semi-circular
field-of-view
j=0;
k=0;
l=0;
deltaG=0;
minD=15;
MAXG= 90*pi/180;
RATEG= 50*pi/180;
dx= lm(1,:) - x(1);
dy= lm(2,:) - x(2);
d2=dx.^2+dy.^2;
d=sqrt(d2);
ang=pi_to_pi(atan2(dy,dx)-x(3));
ang2=pi_to_pi(atan2(dy,dx)-(x(3)+pi/3));
ang3=pi_to_pi(atan2(dy,dx)-(x(3)-pi/3));

ii= find((abs(d) < rmax)&   (abs(d)>5) & (abs(ang)<pi/15 | abs(ang2)<pi/15
| abs(ang3)<pi/15));


j=find(abs(d)>18 & abs(d)< 20 & abs(ang)<pi/15 );
k=find(abs(d) >18 & abs(d)< 20 & abs(ang2)<pi/15 );
l=find(abs(d)>18 & abs(d)< 20 & abs(ang3)<pi/15 );
```

```
if j>0
    if k>0
       if l>0
        deltaG=-pi/2;
       else
        deltaG=-pi/4;
       end
    elseif l>0
        deltaG=0;
    else
        deltaG=-pi/6;
    end
else
    if k>0
        if l>0
            deltaG=+pi/2;
        else
            deltaG=pi/4;
        end

    elseif l>0
    deltaG=pi/6;
    else
        deltaG=0;
    end
end

maxDelta= RATEG*dt;
if abs(deltaG) > maxDelta
    deltaG= sign(deltaG)*maxDelta;
end

% limit angle
g= g+deltaG;
if abs(g) > MAXG
    g= sign(g)*MAXG;
end
x=vehicle_model(x,4,g,4,dt);

lm= lm(:,ii);
idf= idf(ii);

%
%

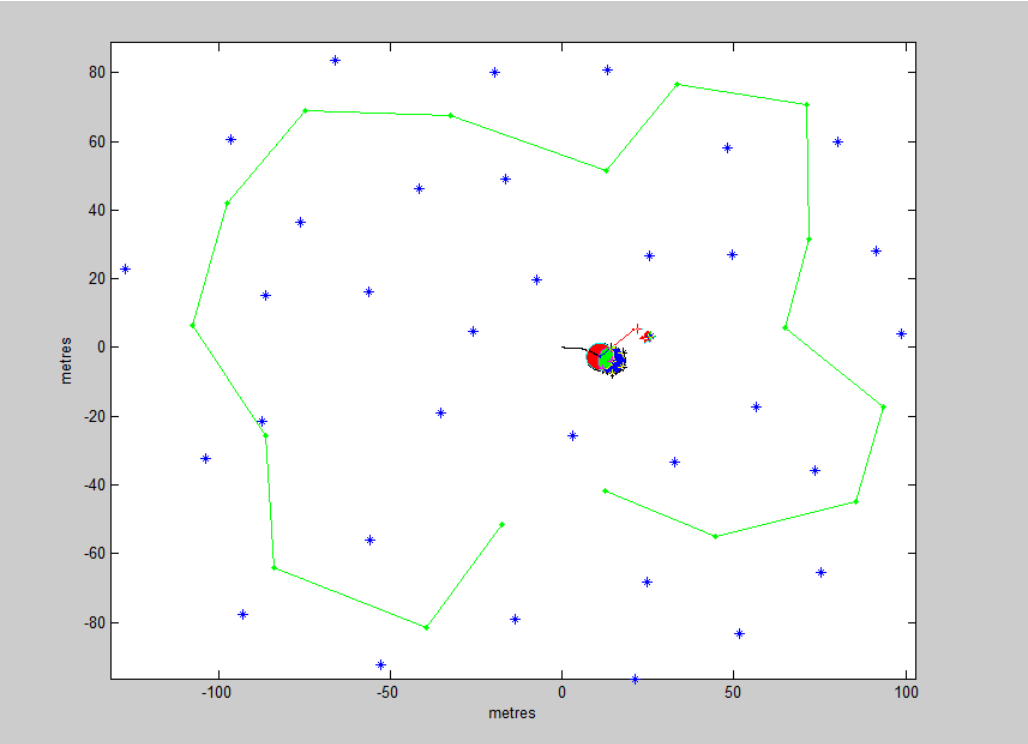
function z= compute_range_bearing(x,lm)
% Compute exact observation
dx= lm(1,:) - x(1);
dy= lm(2,:) - x(2);
phi= x(3);
z= [sqrt(dx.^2 + dy.^2);
    atan2(dy,dx) - phi];
%
%
```

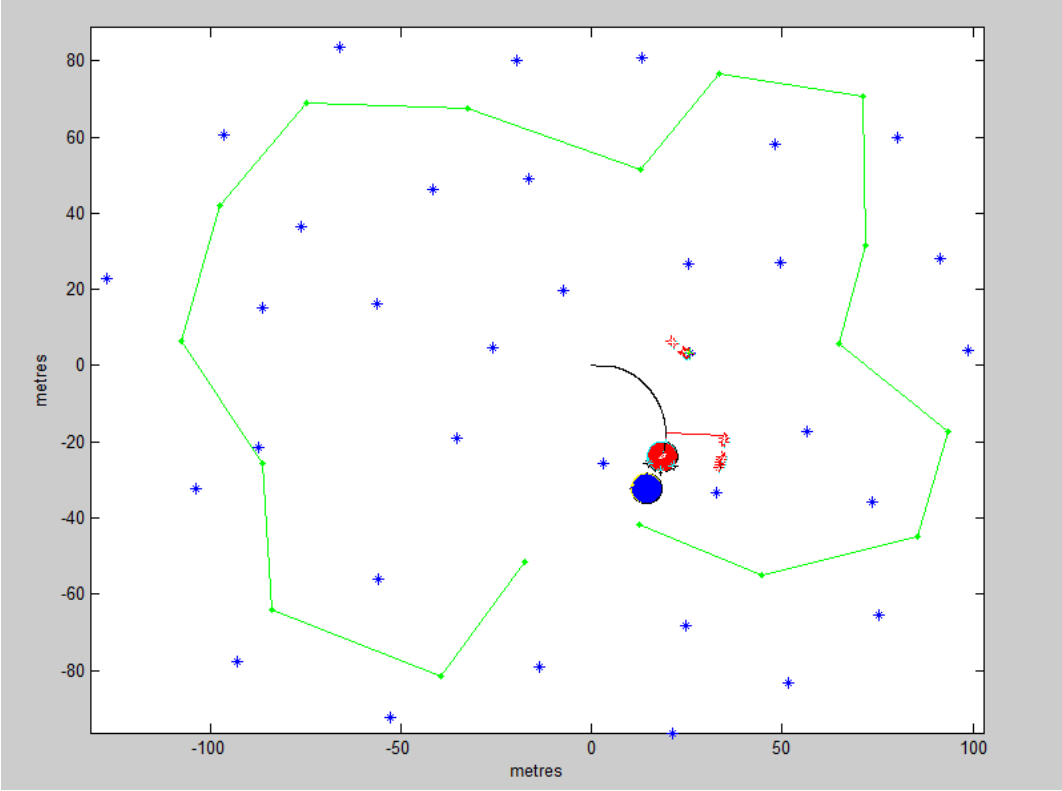
RESULT AND INFERENCE

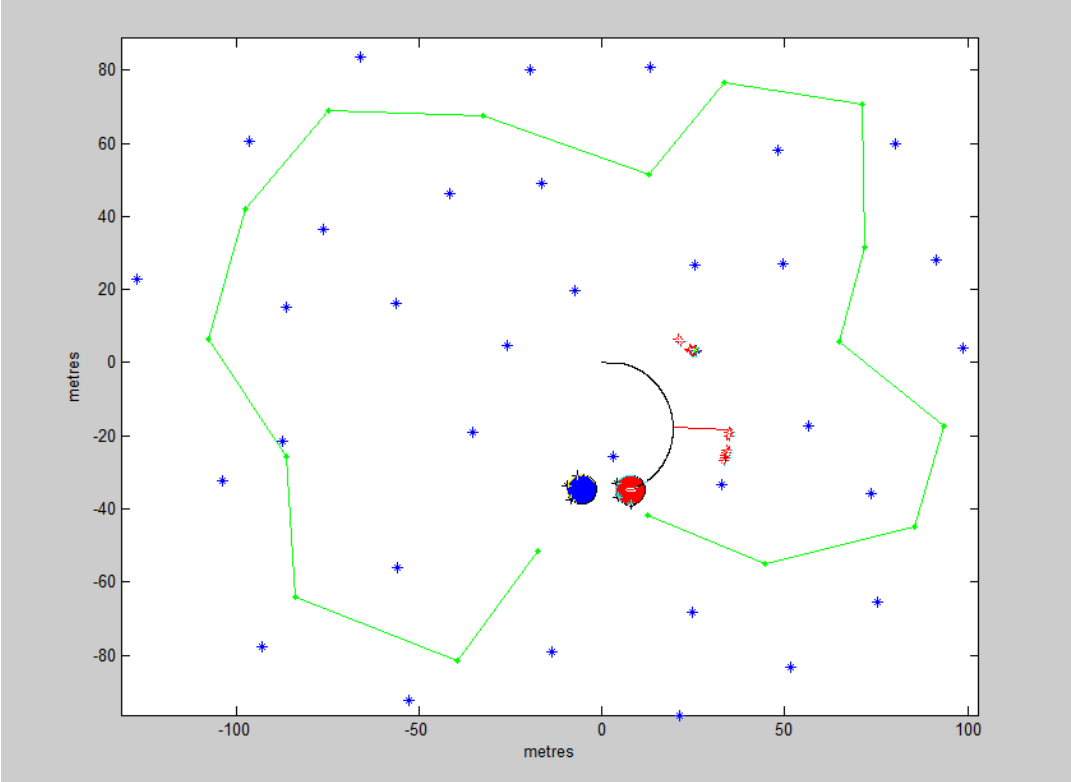
The simulation was run and it was observed that the bot senses the landmarks and moved accordingly but doing so also increased the uncertainty even in the path of the bot as particular anchor points weren't given in this. The bot moved such that the landmark wasn't anywhere near all three sensors. This simulation showed the EKF estimation of the SLAM

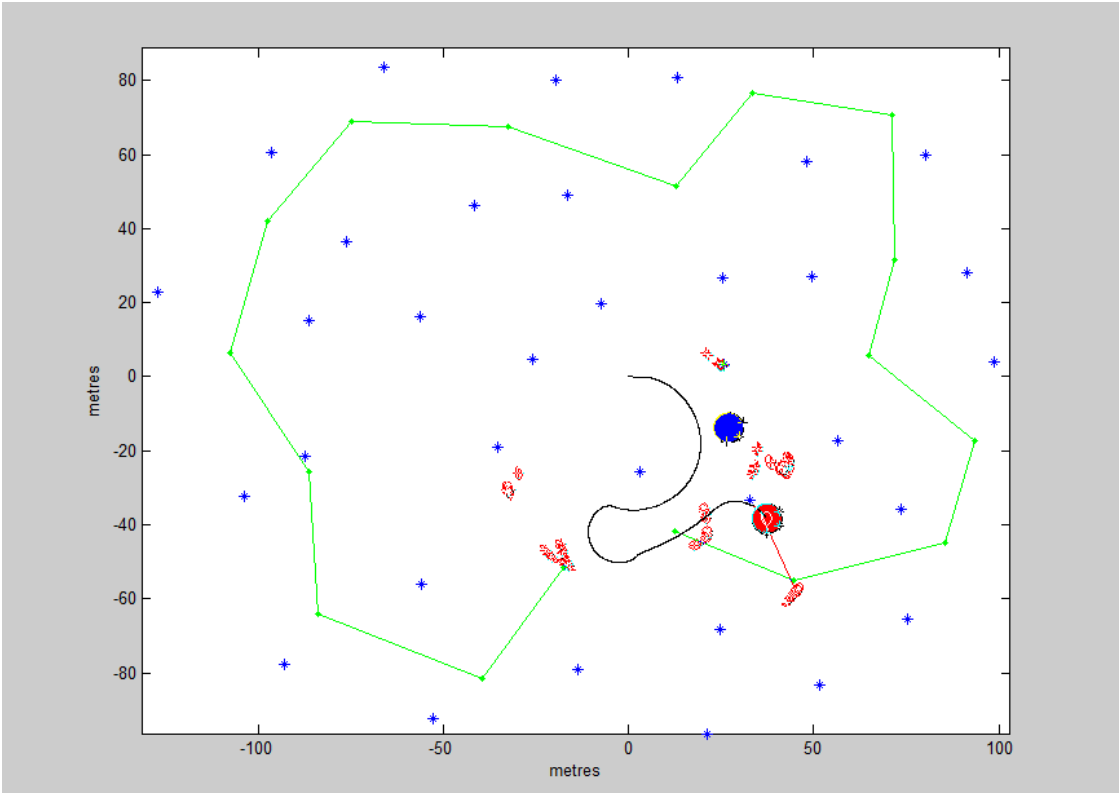
SNAPS FROM SIMULATION

The Blue circle is the true position of the bot.
The Red circle is the predicted position of the bot.
The Blue points are the landmarks in the map.
The Black line is the predicted path.




80
60
40
20
0
-20
-40
-60
-80
metres
-100
-50
0
50
100
metres


80
60
40
20
0
-20
-40
-60
-80
metres
-100
-50
0
50
100
metres



OVERALL INFERENCE

Apart from the above inferences made, it was also observed that

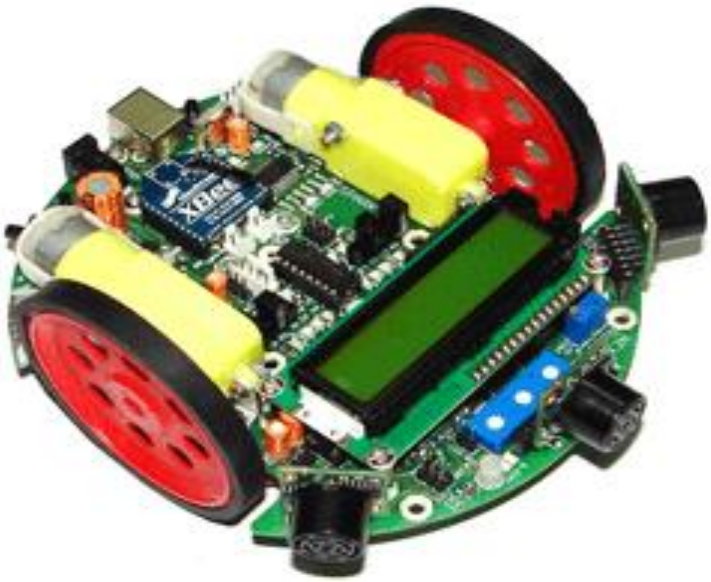
- Decrease in velocity of the bot decreased the uncertainty and sticks close to the path if given.
- Bigger the bot size makes the certainty of the bot clearer.
- Increasing the maximum angle, it can rotate, made the bot freely to rotate and map the area and to decrease the uncertainty.

SECTION III

REAL LIFE NAVIGATION OF THE BOT

About Spark V

The bot that we used for navigation is Spark-V from NEX Robotics which works under ATmega16. Robot comes with rechargeable 7.2V
600mA NiMH Battery and on board intelligent battery charger. It has 3 analog white line sensors, 3 analog IR Proximity sensors, 3 directional light intensity sensors, battery voltage sensing,TSOP1738 IR receiver for TV remote control and Position encoders. Robot has support for 3 MaxBotix EZ series ultrasonic range sensors. It also has support for the servo mounted sensor pod which can be used to make 180 degrees scan for the map making. Robot is powered by 6 cell 7.2V 600mA rechargeable NiMH batteries which gives about one hour battery operation. Robot has built-in Smart Battery Controller which charges the battery in intelligent way and also monitors the battery charge level when robot is in operation. Robot has 2x16 alphanumeric LCD, Lots of LED indicators for quick debugging, Buzzer etc. Motors are controlled by L293D motor driver. Robot gives top speed of 15cm to 20cm per second depending on the model.



Robot has USB interface for PC connectivity. Robot has onboard socket for XBee wireless module for multi robot and robot to PC communication.

Specifications of Spark-V

1) **Sensors:**

Three white line sensors
Three IR proximity sensors
Three directional light intensity sensors
Two Position Encoders
MaxBotix EZ series ultrasonic range sensors (optional)
Servo mounted Ultrasonic Range Sensor (optional)
Battery voltage sensing

2) **Indicators:**

2 x 16 Characters LCD
Indicator LEDs
Buzzer
Battery low indication

3) **Dimensions:**

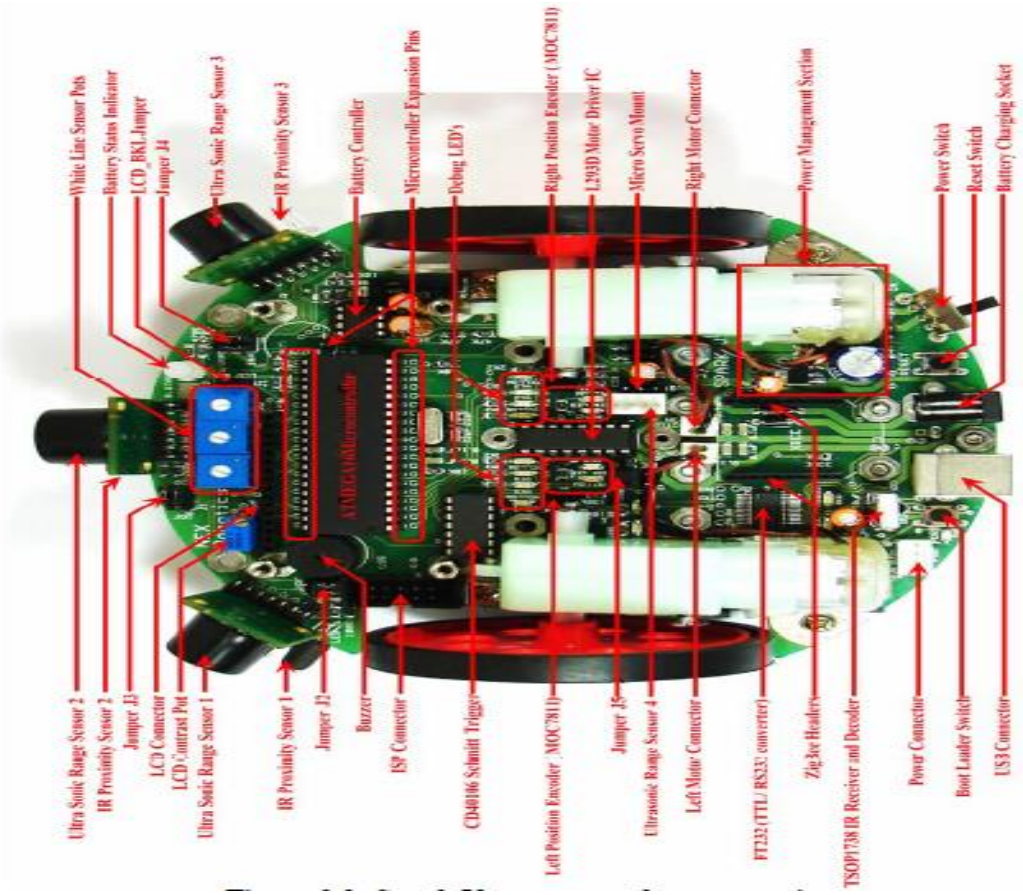
Diameter: 15cm
Height: 7cm

4) **Power:**

6 cell 7.2V 600mA rechargeable NiMH batteries (Batteries included)
On board Smart Battery Controller charges the battery in intelligent way and also monitors the battery charge level when robot is in operation.

5) **Optional Accessories**:

Servo mounted Ultrasonic range sensor for 180 degree scan
Servo mounted directional light intensity sensor for 180 degree scan
MaxBotix EZ series ultrasonic range sensors
XBee wireless module.



Navigation

Here the bot would move using the motors and would sense the landmark through ultrasonic sensor. If the bot finds any landmark in front of it the it would move right left and move where there are no landmarks.

Program for navigation

```
#include <avr/io.h>
#include <avr/interrupt.h>
#include <util/delay.h>

#include <math.h> //included to support power function
#include "lcd.c"

unsigned char ADC_Conversion(unsigned char);
unsigned char ADC_Value;
unsigned int BATT_Voltage;
unsigned char LuS;
unsigned char CuS;
unsigned char RuS;
unsigned long int ShaftCountLeft = 0; //to keep track of left position encoder
unsigned long int ShaftCountRight = 0; //to keep track of right position encoder
```

```
#define read_eeprom_word(address) eeprom_read_word ((const uint16_t*)address)
#define write_eeprom_word(address,value)eeprom_write_word((uint16_t*)address,(uint16_t)value)
#define update_eeprom_word(address,value) eeprom_update_word ((uint16_t*)address,(uint16_t)value)


unsigned int EEMEM  my_eeprom_array[10];


unsigned int my_ram_array[10];

//Function to configure LCD port
void lcd_port_config (void)
{
 DDRC = DDRC | 0xF7;    //all the LCD pin's direction set as output
 PORTC = PORTC & 0x80;  // all the LCD pins are set to logic 0 except PORTC 7
}

//ADC pin configuration
void adc_pin_config (void)
{
 DDRA = 0x00;  //set PORTF direction as input
 PORTA = 0x00; //set PORTF pins floating
}

void ultrsonic_trigger_config(void)
{
 DDRD = DDRD | 0x40;   //all the LCD pin's direction set as output
 PORTD = PORTD & 0x00; // all the LCD pins are set to logic 0 except PORTC 7
}
void left_encoder_pin_config (void)
{
 DDRD  = DDRD & 0xF7;  //Set the direction of the PORTD 3 pin as input
 PORTD = PORTD | 0x08; //Enable internal pull-up for PORTD 3 pin
}
//Function to configure INT0 (PORTD 2) pin as input for the right position encoder
void right_encoder_pin_config (void)
{
 DDRD  = DDRD & 0xFB;  //Set the direction of the PORTD 2 pin as input
 PORTD = PORTD | 0x04; //Enable internal pull-up for PORTD 2 pin
}


//Function to Initialize PORTs
void port_init()
{

 lcd_port_config();
 adc_pin_config();
 ultrsonic_trigger_config();
 DDRB = DDRB | 0x0F;   //set direction of the PORTB3 to PORTB0 pins as output
 PORTB = PORTB & 0xF0; // set initial value of the PORTB3 to PORTB0 pins to logic 0
 DDRD = DDRD | 0x30;   //Setting PD4 and PD5 pins as output for PWM generation
 PORTD = PORTD | 0x30;
left_encoder_pin_config();    //left encoder pin config
right_encoder_pin_config();

}
void left_position_encoder_interrupt_init (void) //Interrupt 1 enable
{
 cli(); //Clears the global interrupt
```

```
 MCUCR = MCUCR | 0x08; // INT1 is set to trigger with falling edge
 GICR = GICR | 0x80;   // Enable Interrupt INT1 for left position encoder
 sei(); // Enables the global interrupt
}

void right_position_encoder_interrupt_init (void) //Interrupt 0 enable
{
 cli(); //Clears the global interrupt
 MCUCR = MCUCR | 0x02; // INT0 is set to trigger with falling edge
 GICR = GICR | 0x40;   // Enable Interrupt INT5 for right position encoder
 sei(); // Enables the global interrupt
}
ISR(INT0_vect)
{
 ShaftCountRight++;  //increment right shaft position count
}

//ISR for left position encoder
ISR(INT1_vect)
{
 ShaftCountLeft++;  //increment left shaft position count
}


//Function to Initialize ADC
void adc_init()
{
 ADCSRA = 0x00;
 ADMUX = 0x20;                  //Vref=5V external --- ADLAR=1 --- MUX4:0 = 0000
 ACSR = 0x80;
 ADCSRA = 0x86;                 //ADEN=1 --- ADIE=1 --- ADPS2:0 = 1 1 0
}

//This Function accepts the Channel Number and returns the corresponding Analog Value
unsigned char ADC_Conversion(unsigned char Ch)
{
 unsigned char a;
 Ch = Ch & 0x07;
 ADMUX= 0x20| Ch;
 ADCSRA = ADCSRA | 0x40;               //Set start conversion bit
 while((ADCSRA&0x10)==0);    //Wait for ADC conversion to complete
 a=ADCH;
 ADCSRA = ADCSRA|0x10;         //clear ADIF (ADC Interrupt Flag) by writing 1 to it
 return a;
}

// This Function prints the Analog Value Of Corresponding Channel No. at required Row
// and Coloumn Location.

// Ultrasonic sensor are connected in chaining mode. This function rise a
// trigger pulse of >20usec to command ringing.
void ultrasonic_trigger(void)
{
 PORTD = PORTD | 0x40;  // make high the Trigger input for Ultrasonic sensor
 _delay_us(50);        // Wait for >20usec
 PORTD = PORTD & 0xBF;  // make low the Trigger input for Ultrasonic sensor
}

void init_devices (void)
{
```

```
 cli();          //Clears the global interrupts
 port_init();
 adc_init();
left_position_encoder_interrupt_init();
right_position_encoder_interrupt_init();

 sei();          //Enables the global interrupts
}

//Main Function
int main(void)
{
 init_devices();
 int i=0,j=0,k=0;
 float l1[i],l2[i],r1[j],r2[j],c1[k],c2[k];
 float r=2.28;
 lcd_set_4bit();
 lcd_init();
 while(1)
 {
            ultrasonic_trigger();     // call ultrasonic triggering after enery 150msec
   _delay_ms(150);

   LuS = ADC_Conversion(0) * 2;        // In Inches
   CuS = ADC_Conversion(1) * 2;       // In Inches
   RuS = ADC_Conversion(2) * 2;

            if(LuS<12)
            {
    PORTB=0b00000010;
             }
            if(RuS<12)
            {
                        PORTB=0b00000100;
            }


            if(CuS<12)
            {
   if(LuS>12)
                        {
                                    PORTB=0b00000010;
                        }
            else if(LuS<12)
                        {
      PORTB=0x00;
                        }
            }
            else
            {
                        PORTB=0b00000110;
            }

   lcd_print(1,1,LuS,3);
   lcd_print(1,5,CuS,3);
   lcd_print(1,9,RuS,3);
   lcd_print(2,1,ShaftCountLeft,3);
   lcd_print(2,5,ShaftCountRight,3);
```

```
        update_eeprom_word(&my_eeprom_array[0], LUS);  // write value 1 to position 0 of the eeprom array
   update_eeprom_word(&my_eeprom_array[1], CUS);  // write value 2 to position 1 of the eeprom array
   update_eeprom_word(&my_eeprom_array[2], RUS);

        my_ram_array[0] = read_eeprom_word(&my_eeprom_array[0]);   // read value from position 0 of the
eeprom array to ram array position 0
   my_ram_array[1] = read_eeprom_word(&my_eeprom_array[1]);   // read value from position 1 of the eeprom
array to ram array position 1
   my_ram_array[2] = read_eeprom_word(&my_eeprom_array[2]);
        if(LuS<12)
   {l1[i]=(r+LuS)*cos(45);
        l2[i]=(r+LuS)*sin(45);
        i=i+1;
        }
        if(RuS<12)
   {r1[j]=(r+RuS)*cos(45);
        r2[j]=(r+RuS)*sin(45);
        j=j+1;
        }
        if(CuS<12)
        {c1[k]=(r+CuS)*cos(45);
        c2[k]=(r+CuS)*sin(45);
        k=k+1;
        }

   //BATT_Voltage = (ADC_Conversion(6) * 0.03921) + 0.7;          //Prints Battery Voltage Status
   //lcd_print(2,13,BATT_Voltage,4);
   //Prints Battery Voltage Status
  }

}
```

Code for lcd display

```
#include <avr/io.h>
#include <avr/delay.h>
#include <util/delay.h>

#define RS 0
#define RW 1
#define EN 2
#define lcd_port PORTC

#define sbit(reg,bit)          reg |= (1<<bit)
#define cbit(reg,bit)          reg &= ~(1<<bit)

void init_ports();
void lcd_reset_4bit();
void lcd_init();
void lcd_wr_command(unsigned char);
void lcd_wr_char(char);
void lcd_home();
void lcd_cursor(char, char);
void lcd_print(char, char, unsigned int, int);
void lcd_string(char*);

unsigned int temp;
unsigned int unit;
```

```
unsigned int tens;
unsigned int hundred;
unsigned int thousand;
unsigned int million;

int i;


//Function to Reset LCD
void lcd_set_4bit()
{
	_delay_ms(1);

	cbit(lcd_port,RS);			//RS=0 --- Command Input
	cbit(lcd_port,RW);			//RW=0 --- Writing to LCD
	lcd_port = 0x30;			//Sending 3 in the upper nibble
	sbit(lcd_port,EN);			//Set Enable Pin
	_delay_ms(5);				//delay
	cbit(lcd_port,EN);			//Clear Enable Pin

	_delay_ms(1);

	cbit(lcd_port,RS);			//RS=0 --- Command Input
	cbit(lcd_port,RW);			//RW=0 --- Writing to LCD
	lcd_port = 0x30;			//Sending 3 in the upper nibble
	sbit(lcd_port,EN);			//Set Enable Pin
	_delay_ms(5);				//delay
	cbit(lcd_port,EN);			//Clear Enable Pin

	_delay_ms(1);

	cbit(lcd_port,RS);			//RS=0 --- Command Input
	cbit(lcd_port,RW);			//RW=0 --- Writing to LCD
	lcd_port = 0x30;			//Sending 3 in the upper nibble
	sbit(lcd_port,EN);			//Set Enable Pin
	_delay_ms(5);				//delay
	cbit(lcd_port,EN);			//Clear Enable Pin

	_delay_ms(1);

	cbit(lcd_port,RS);			//RS=0 --- Command Input
	cbit(lcd_port,RW);			//RW=0 --- Writing to LCD
	lcd_port = 0x20;			//Sending 2 in the upper nibble to initialize LCD 4-bit mode
	sbit(lcd_port,EN);			//Set Enable Pin
	_delay_ms(5);				//delay
	cbit(lcd_port,EN);			//Clear Enable Pin
}

//Function to Initialize LCD
void lcd_init()
{
	_delay_ms(1);

	lcd_wr_command(0x28); //4-bit mode and 5x8 dot character font
	lcd_wr_command(0x01); //Clear LCD display
	lcd_wr_command(0x06); //Auto increment cursor position
	lcd_wr_command(0x0E); //Turn on LCD and cursor
	lcd_wr_command(0x80); //Set cursor position
}
```

```
//Function to write command on LCD
void lcd_wr_command(unsigned char cmd)
{
	unsigned char temp;
	temp = cmd;
	temp = temp & 0xF0;
	lcd_port &= 0x0F;
	lcd_port |= temp;
	cbit(lcd_port,RS);
	cbit(lcd_port,RW);
	sbit(lcd_port,EN);
	_delay_ms(5);
	cbit(lcd_port,EN);

	cmd = cmd & 0x0F;
	cmd = cmd<<4;
	lcd_port &= 0x0F;
	lcd_port |= cmd;
	cbit(lcd_port,RS);
	cbit(lcd_port,RW);
	sbit(lcd_port,EN);
	_delay_ms(5);
	cbit(lcd_port,EN);
}

//Function to write data on LCD
void lcd_wr_char(char letter)
{
	char temp;
	temp = letter;
	temp = (temp & 0xF0);
	lcd_port &= 0x0F;
	lcd_port |= temp;
	sbit(lcd_port,RS);
	cbit(lcd_port,RW);
	sbit(lcd_port,EN);
	_delay_ms(5);
	cbit(lcd_port,EN);

	letter = letter & 0x0F;
	letter = letter<<4;
	lcd_port &= 0x0F;
	lcd_port |= letter;
	sbit(lcd_port,RS);
	cbit(lcd_port,RW);
	sbit(lcd_port,EN);
	_delay_ms(5);
	cbit(lcd_port,EN);
}


void lcd_home()
{
	lcd_wr_command(0x80);
}


//Function to Print String on LCD
void lcd_string(char *str)
```

```
{
	while(*str != '\0')
	{
		lcd_wr_char(*str);
		str++;
	}
}

//Position the LCD cursor at "row", "column"

void lcd_cursor (char row, char column)
{
	switch (row) {
		case 1: lcd_wr_command (0x80 + column - 1); break;
		case 2: lcd_wr_command (0xc0 + column - 1); break;
		case 3: lcd_wr_command (0x94 + column - 1); break;
		case 4: lcd_wr_command (0xd4 + column - 1); break;
		default: break;
	}
}

//Function to print any input value up to the desired digit on LCD
void lcd_print (char row, char coloumn, unsigned int value, int digits)
{
	unsigned char flag=0;
	if(row==0||coloumn==0)
	{
		lcd_home();
	}
	else
	{
		lcd_cursor(row,coloumn);
	}
	if(digits==5 || flag==1)
	{
		million=value/10000+48;
		lcd_wr_char(million);
		flag=1;
	}
	if(digits==4 || flag==1)
	{
		temp = value/1000;
		thousand = temp%10 + 48;
		lcd_wr_char(thousand);
		flag=1;
	}
	if(digits==3 || flag==1)
	{
		temp = value/100;
		hundred = temp%10 + 48;
		lcd_wr_char(hundred);
		flag=1;
	}
	if(digits==2 || flag==1)
	{
		temp = value/10;
		tens = temp%10 + 48;
		lcd_wr_char(tens);
		flag=1;
	}
```

```
        if(digits==1 || flag==1)
        {
                unit = value%10 + 48;
                lcd_wr_char(unit);
        }
        if(digits>5)
        {
                lcd_wr_char('E');
        }
}
```

About the code for navigation

1) In the code first we are giving a input impulse with a time period of 20microsecond to the trigger pin of ultrasonic sensor. The ultrasonic sensor gives an output of 9.8mV per inches so as the and it gives an output of 1inch resolution so when we move to ADC conversion we would get only the even number of distances as we are multiplying the digital value to 2. After we did with how to find the range we kept a maximum range and if the range reaches to its value the bot will move right or left. For the working of the bot it needs three ultrasonic sensor.

2) After finding the range we would find the position of the bot in the map by counting the number of slotted disc the has entered the position encoder.

   Position encoder

   Position encoders give position / velocity feedback to the robot. It is used in closed loop to control robot's position and velocity. Position encoder consists of slotted disc which rotates between optical encoder (optical transmitter and receiver). When slotted disc moves in between the optical encoder we get square wave signal whose pulse count indicates position and time period / frequency indicates velocity.

3) Third step is finding the landmark coordinates by using property of circle and then we are adding it in an array and storing it into the eeprom of ATmega16 so that we can we know the position of the bot in the map.

4) The LCD code is for displaying the shaft count, the range of the landmark, and position of the landmark.

RESULT AND INFERENCE

The code was programmed into the bot which was mounted with ultrasonic sensors and it was made to run in a room filled with obstacles and it traversed in this room without colliding with any of the obstacles and implemented the navigation code.

CONCLUSION

In the duration of two months, we learnt SLAM and understood the basic concept of it. We learnt in detail about the EKF SLAM and did various methods to implement it. Initially we started with the simulation of a bot undergoing EKF SLAM. Later we modified this simulation and did a better simulation of self navigation. In the end we implemented this in a real life navigation of a bot by programming a code into a SPARK V bot to navigate using SLAM.

REFRENCES

- Link to video lecture series: https://www.youtube.com/watch?v=U6vr3iNrwRA&list=PLgnQpQtFTOGQrZ4O5QzbIHgl3b1JHimN
- Source materials used for learning SLAM
  1. https://ocw.mit.edu/courses/aeronautics-and-astronautics/16-412j-cognitive-robotics-spring-2005/projects/1aslam_blas_repo.pdf
  2. http://ais.informatik.uni-freiburg.de/teaching/ws12/mapping/pdf/slam04-ekf-slam.pdf
  3. http://www.iri.upc.edu/people/jsola/JoanSola/objectes/curs_SLAM/SLAM2D/SLAM%20course.pdf
- MATLAB simulation reference: http://www-personal.acfr.usyd.edu.au/tbailey/
- Learning about AVR: http://maxembedded.com/category/microcontrollers-2/atmel-avr/